Layer Distinctions: Why CKS Is Not a Workflow Engine, an Agent Framework, or a Control Plane

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: 29 April 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its contribution is to articulate the architectural boundary between CKS and three families of adjacent design objects — workflow engines, agent frameworks, and control planes — by extending the layer-distinction framing of the source paper (the substrate-vs-cell split at §2.1, the four-layer landscape at §4.3, the MACI treatment at §4.4, and the Stevens complementarity claim at §8.1) to cover workflow engines and agent frameworks specifically. The control-plane treatment recapitulates the source paper directly; the workflow-engine and agent-framework treatments name the extension explicitly where it occurs.

Abstract

The Coordination Knowledge Substrate (CKS) pattern is at risk of being conflated with three categories of adjacent design object: workflow engines (BPMN-based systems, code-defined orchestrators, scheduling and no-code automators), agent frameworks (multi-agent libraries built around LLM execution), and control planes (Stevens-style operational governance primitives for AI agents). Each shares surface properties with CKS — they coordinate work, involve LLMs, preserve some kind of state — but each commits at a different architectural layer. This note formalizes a three-layer model implicit in the source paper: a coordination-knowledge layer (CKS), a coordination-mechanism layer (workflow engines and agent frameworks), and a control-plane layer (Stevens-style primitives). It names the boundary, the most common conflation, and an operational test for each, and describes the composition pattern by which a complete coordination system uses all three layers without any one substituting for another. The control-plane treatment recapitulates the source paper directly; the workflow-engine and agent-framework treatments extend its layer-distinction framing.

1. Why the layer distinction needs to be named

The source paper distinguishes CKS from five neighbors at §2.3 — RAG, parametric memory, multi-agent protocols, governance platforms, and complete system architectures — and treats control-plane work as a complementarity claim at §4.3, §4.4, and §8.1. Two object families commonly conflated with CKS in marketplace discussions of AI coordination are not separately addressed: workflow engines and agent frameworks. The underlying layer model that the complementarity claim sits inside is also not rendered as a single explicit object. This note fills both gaps.

The conflations matter because the three adjacent families share surface properties with CKS: they coordinate work, many of their current deployments involve LLMs, and they preserve some kind of state across operations. The architectural distinction is not about what they do on the surface; it is about what each layer is responsible for. Naming the distinction is what makes hybrid deployments coherent — a CKS substrate fronted by a workflow engine, a cell triggered by an agent framework, a deployment hosted under a control plane — and prevents the misreading that CKS competes with these objects rather than composing with them.

The treatment of workflow engines (§3) and agent frameworks (§4) extends the source paper's layer-distinction framing — §2.1 and §4.4 — to design-object families the source paper does not directly name. The treatment of control planes (§5) recapitulates §4.3 and §8.1 directly and does not extend them.

2. The three architectural layers

A complete coordination system typically involves all three layers, but no single layer is responsible for what the others handle. CKS commits at one of the three.

The coordination-knowledge layer. Where coordination state lives — what was decided, by whom, under what authority, with what rationale, and what contradictions remain unresolved. The CKS substrate operates here. Its commitments are about state: persistent, conflict-preserving, human-governed in the authority sense formalized at §2.1 and §3.3, and addressable across sessions. The cell layer extends the same commitments to functional units: orchestration rules over the substrate are human-authored, and cell behavior follows those rules at execution time.

The coordination-mechanism layer. Where work executes — where steps run, routing happens, retries are triggered, parallelism is managed, and (for agent frameworks) multi-agent communication and tool calling are structured. Workflow engines and agent frameworks operate here. Their commitments are about behavior: step sequences, transitions, execution semantics, and the LLM execution machinery that produces them. Source paper §4.4 treats MACI as the canonical 2024–2026 example of a coordination-mechanism-layer commitment in the multi-agent direction; this note extends that framing to the broader class of orchestration-layer objects.

The control-plane layer. Where infrastructure-level concerns are managed — process lifecycle, resource allocation, identity, access control, observability. Control-plane primitives, in the sense Stevens uses the term in his work on trustworthy AI agents (cited at §8.1), operate here. Their commitments are about operations: lifecycle, authentication, authorization, audit logging at the runtime level, and the gateway-layer policies that

make agent operation governable in the operational sense.

Source paper §4.3 renders this as a four-layer landscape (reasoning execution, coordination mechanisms, control plane, coordination knowledge). This note collapses §4.3's reasoning-execution layer into the broader coordination-mechanism layer for §3 and §4, because workflow engines and agent frameworks include both concerns within a single class of design object. The collapse is for expository convenience and does not relocate any commitment.

3. Why CKS is not a workflow engine

The treatment in this section extends the source paper's layer-distinction framing — §2.1 and §4.4 — to a design-object family the source paper does not directly name.

What workflow engines are. Workflow engines coordinate the execution of multi-step processes against a process specification: they run the steps in order, handle branching and retries, coordinate parallel branches, and persist execution state across long-running processes. The category includes BPMN-based systems, code-defined orchestrators, scheduling systems, and no-code automators. They are coordination-mechanism-layer objects that define and enforce step sequences.

The architectural difference. Workflow engines persist execution state — which step ran, what its inputs and outputs were, what errors occurred. They do not commit to coordination-knowledge state. A workflow engine can run a process for years and faithfully record every step that ran without ever answering *what was the rationale for the decision this process implements*. CKS substrates carry that state by design: substrate content is decisions, rationale, authority, and preserved conflict, governed by the cell's orchestration rules. The two kinds of state are different in kind, not different in degree.

The most common conflation. Treating a workflow engine's execution state as if it were coordination-knowledge state. Workflow execution logs are accountability traces in a narrow sense — they record what the engine did — but not the trace CKS specifies (what was decided, under what authority, with what rationale). A workflow engine can support a CKS deployment by triggering cells that read from and write to a substrate; it cannot replace the substrate, because its design does not commit to carrying coordination-knowledge state.

Operational test. If a deployment answers "what was decided and why" by reading workflow execution state, the engine is being used as a substrate substitute and the substrate's commitments are violated. If the deployment answers by reading from a substrate that the engine wrote into (or from which the engine triggered cells that wrote into the substrate), the layers are correctly separated.

4. Why CKS is not an agent framework

The treatment in this section extends the source paper's layer-distinction framing — §2.1, the AI-as-substrate-mediator commitment at §4.2, and the MACI treatment at §4.4 — to a design-object family the source paper does not directly name. This is the conflation most commonly raised in current marketplace discussions, because agent-framework memory and CKS substrate look superficially similar in a way that workflow execution state and CKS substrate do not.

What agent frameworks are. Agent frameworks are coordination-mechanism-layer objects specialized for

LLM-driven execution: they define how LLM agents communicate, how planning and reflection are handled, how tool calls are routed, and how multi-step reasoning is structured into a system-level result. They typically include some form of memory as part of the execution machinery — per-session conversation state, per-agent scratchpads, vector stores over past interactions, summary stores for long-running agents, and shared memories accessible across agents inside a single framework instance.

The architectural difference. Agent frameworks persist execution state for the duration of an agent's run: agent states, message histories, tool-call records, and the memory store the framework provides. The state they hold is the agent's, organized for the agent's consumption, written and read primarily by the agent or by other agents inside the same framework. When the agent terminates, its state is discarded, archived in a framework-level store, or summarized into a memory representation that lives at the framework's layer. CKS substrates persist coordination state independent of any agent's lifecycle, and the AI-as-substrate-mediator commitment at §4.2 specifies that LLMs do not hold substrate-relevant state outside the substrate. Agent-framework memory is execution state that happens to persist; CKS substrate is coordination state that humans govern.

The most common conflation. Treating agent-framework memory as if it were a CKS substrate. The two surfaces look similar — both store content related to LLM operations, both can be read across sessions, both can grow over time — but they differ on every architectural commitment that matters. Agent memory is execution state with a persistence feature; CKS substrate is coordination state with a governance commitment. Agent memory is read primarily by the agent that wrote it; CKS substrate is read by humans exercising governance and by cells executing under human-authored orchestration rules. Agent memory sits behind the framework's interface and is shaped by the framework's data model; CKS substrate is human-inspectable and human-modifiable in the tools it is hosted in. The two patterns can coexist — an agent reads from a substrate, an agent writes to a substrate under orchestration rules — but they are architecturally distinct objects, and the coexistence is a composition, not an identity.

Operational test. If a system's "memory" is held inside an agent framework's execution context, organized for agent consumption, and read primarily by agents within that framework, it is agent-framework memory. If the same content is held in a substrate humans can directly read and write under governance authority, addressable across sessions independent of any agent's lifecycle, it is CKS substrate. Two further properties sharpen the test: whether a human can modify the content directly without the framework's mediation (if not, framework memory), and whether the content persists after every agent terminates by architectural commitment rather than by archival convention (if conditional, framework memory; if guaranteed, CKS substrate).

5. Why CKS is not a control plane

The treatment in this section recapitulates source paper §4.3 and §8.1 and does not extend them.

What control planes are. Control planes manage the operational concerns of a running coordination system: process lifecycle, resource allocation, identity, access control, observability, and the gateway-layer policies that determine who or what is permitted to invoke which capability under which conditions. Stevens's work on trustworthy AI agents (referenced at §8.1) treats control-plane primitives — approval gates, reviewer

workflows, accountable decision logs, break-glass override — as the primitives that make autonomous-agent deployment governable in the operational sense.

The architectural difference. Control planes answer infrastructure questions about whether and how the system runs; they do not answer coordination questions about what the system is doing or why. Per §8.1, CKS is complementary to control-plane primitives: the control plane provides the infrastructure that makes a substrate and its cells operationally viable, and the substrate provides the coordination knowledge the running system operates on.

The most common conflation. Treating control-plane authority decisions (who can call which API, who is authenticated to which environment) as if they were CKS authority decisions (the inspect, modify, and override rights formalized in the human-governed commitment). Control-plane access controls govern who can read or write to the substrate's host environment; CKS authority decisions govern who has the substrate-level rights that make the substrate human-governed in the architectural sense. A user with full control-plane access to the host might still not be a CKS governor. Conversely, a CKS governor must hold control-plane access sufficient to exercise their rights — control-plane access is necessary, but not sufficient, for CKS governance.

6. The composition pattern: how CKS layers with orchestration-layer objects

The three layers compose. A complete deployment may use all three; each layer's role is distinct; the substrate's commitments are not delegated to or claimed from the others. Specific composition primitives — cell-triggering APIs, substrate-aware agent memory interfaces, control-plane policies that read substrate content — are explicitly future work per source paper §13.3 and not the subject of this note. What this note identifies is the general shape.

A workflow engine triggers a CKS cell. The engine decides when the cell should run, given the workflow's process state and triggering conditions; the cell reads from and writes to the substrate under human-authored orchestration rules. Two records exist after the cell runs: the engine records that it ran the cell; the substrate records what the cell decided, under what authority, with what rationale, against what conflicts. Neither substitutes for the other.

An agent framework executes a CKS cell. The framework manages the LLM execution machinery — prompting, tool calling, multi-step reasoning, multi-agent communication where applicable; the cell's role is the same as above. Framework memory holds execution state for the duration of the cell's run; the substrate holds coordination state across cell runs. When the framework's agents terminate, framework memory is discarded or archived at the framework's layer; the substrate persists at its own layer, governed by the rights named in the human-governed commitment.

A control plane authorizes substrate access. The control plane authenticates the user and authorizes the host-level access being exercised; the substrate enforces CKS-level governance authority once that access is granted. Control-plane access is necessary for CKS governance to be exercisable, and CKS governance is the layer at which the substrate's content-level authority decisions are made.

What the three patterns share. Across all three, the substrate is the only layer whose state persists by

architectural commitment across cell runs and across the lifecycles of the other layers' objects. Workflow engines persist execution state for the duration of a process instance; agent frameworks for the duration of an agent run; control planes for the duration of an operational concern. The substrate persists coordination state across all those lifecycles, because coordination state is what makes the work the other layers execute coherent across sessions, agents, and runs. Each layer is responsible for state at its own timescale; the substrate is the only layer responsible for coordination state at the timescale that survives every other object's lifecycle.

7. Operational test

A system maintains the layer distinction if and only if all of the following are true at all times during the substrate's existence:

1. Coordination-knowledge state — decisions, authority, rationale, and preserved conflicts — lives in a substrate, not in workflow execution logs, agent-framework memory, or control-plane state.
2. Execution behavior — step sequences, retries, parallelism, multi-agent coordination, tool routing, and the LLM execution machinery — lives in workflow engines or agent frameworks where appropriate, not in the substrate.
3. Infrastructure concerns — process lifecycle, identity, access control, resource allocation, and operational observability — live in the control plane, not in the substrate.
4. The three layers compose: a complete deployment may use all three, but each layer's role is distinct, the substrate's commitments are not delegated to the other layers, and the other layers' commitments are not claimed for the substrate.

A system that fails any of (1)–(4) may be a useful system, and may govern coordination at some other layer, but it does not maintain the layer distinction the CKS pattern relies on.

8. Why naming this layer distinction matters

Conflations at any of the three layer boundaries produce systems that are coherent at one layer and broken at another. Implementations that conflate CKS with workflow engines build workflow tools that record what ran and lose what was decided. Implementations that conflate CKS with agent frameworks build agent systems that hold rich state during a run and lose coordination state when agents terminate. Implementations that conflate CKS with control planes confuse infrastructure access with governance authority — a user with operational access to the host is not, by virtue of that access, a CKS governor.

The layer model in §2 is not an argument that CKS is better than workflow engines, agent frameworks, or control planes; it is the architectural account of how these design objects fit together when each is doing its own work, and why the substrate's commitments do not transfer to or from the other three. The source paper's positioning of CKS at §2.3 — as a design pattern at the coordination-knowledge layer that composes with reasoning-execution, coordination-mechanism, and control-plane layers rather than replacing them — is the framing this note operationalizes.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Layer Distinctions: Why CKS Is Not a Workflow Engine, an Agent Framework, or a Control Plane.* 29 April 2026. ORCID: 0009-0004-8065-3235.